3ra Lista de Problemas y Ejercicios

Teoría de la Computación

2026/2

Esta lista de problemas y ejercicios debe entregarse en formato PDF y se sugiere que sea realizada en computadora utilizando la herramienta JFLAP para diseñar y simular los autómatas y las expresiones regulares. En la parte superior de cada hoja debe aparecer la información general del alumno (nombre completo, boleta y grupo). Además, debe incluir las instrucciones y/o enunciados de cada ejercicio, no sólo las respuestas. Finalmente, todas las respuestas de los ejercicios deben ir acompañadas del procedimiento y/o razonamiento. **NOTA**: Los ejercicios que debe entregar en esta tarea son los que tienen un asterisco (*). El resto son opcionales.

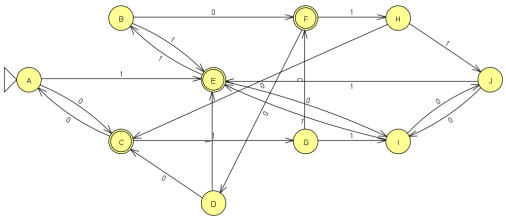
1. Autómatas Finitos y Simplificación

En los problemas 151-155, debe hallar el diagrama de transiciones del autómata definido en cada tabla y simplificarlo eliminando los estados que no son accesibles. A continuación, encontrará las tablas para cada uno de los problemas:

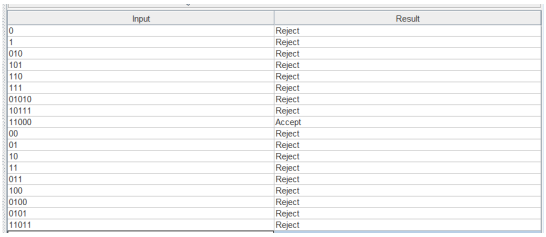
151. *

Estado	0	1
$\rightarrow A$	C	E
B	F	E
$*C$	A	G
D	C	E
$*E$	I	B
$*F$	D	H
G	F	I
H	C	J
I	J	E
J	I	E

Razonamiento: Al trazar los caminos desde el estado inicial A, se comprueba que todos los estados son alcanzables. No existen estados inaccesibles; el autómata no se reduce.



Autómata 151



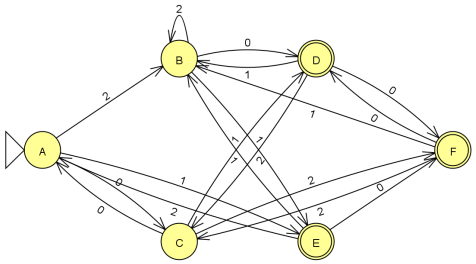
Input	Result
0	Reject
1	Reject
010	Reject
101	Reject
110	Reject
111	Reject
01010	Reject
10111	Reject
11000	Accept
00	Reject
01	Reject
10	Reject
11	Reject
011	Reject
100	Reject
0100	Reject
0101	Reject
11011	Reject

Pruebas 151

152. *

Estado	0	1	2
$\rightarrow A$	C	E	B
B	D	E	B
C	A	D	F
$*E$	F	B	C
$*F$	D	B	C

Razonamiento: Explorando desde el estado inicial A, se alcanza rápidamente el resto de los estados (B, C, D, E, F). No hay estados inaccesibles a eliminar.



Autómata 152

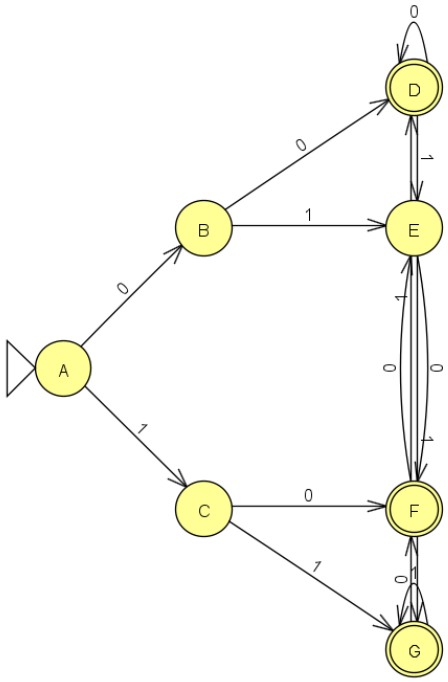
Input	Result
1	Accept
01	Accept
02	Accept
20	Accept
21	Accept
10	Accept
001	Accept
111	Accept
220	Accept
0	Reject
2	Reject
00	Reject
11	Reject
12	Reject
012	Reject
212	Reject
201	Reject
102	Reject

Pruebas 152

153. *

Estado	0	1
$\rightarrow A$	B	C
B	D	E
C	F	G
$*D$	D	E
E	F	G
$*F$	E	D
$*G$	F	G

Razonamiento: Todos los estados del autómata (A, B, C, D, E, F, G) son accesibles partiendo desde el inicial A. El autómata queda igual.



Autómata 153

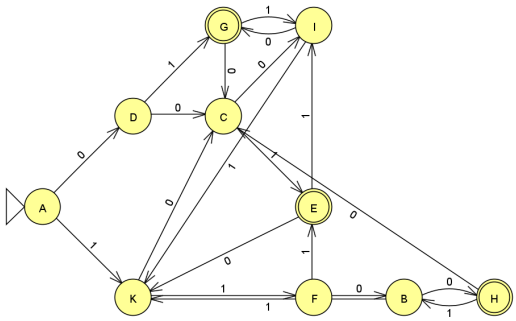
Input	Result
00	Accept
10	Accept
11	Accept
000	Accept
010	Accept
011	Accept
101	Accept
111	Accept
0010	Accept
0	Reject
1	Reject
01	Reject
001	Reject
100	Reject
0100	Reject
1100	Reject
0001	Reject
01100	Reject

Pruebas 153

154. *

Estado	0	1
$\rightarrow A$	D	K
B	H	K
C	I	E
D	C	G
$*E$	K	I
F	B	E
$*G$	C	I
$*H$	C	B
I	G	K
$*J$	F	B
K	C	F

Razonamiento: Al realizar el recorrido exhaustivo desde el estado A, se alcanzan todos los estados a excepción del estado J (la letra J nunca aparece del lado derecho de la tabla). Por lo tanto, J es inaccesible y se elimina del autómata.



Autómata 154 (Simplificado)

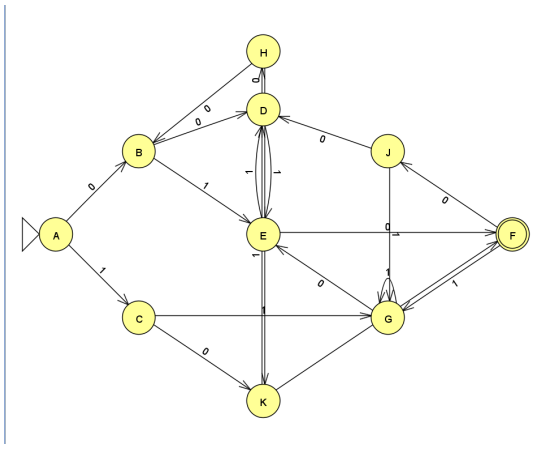
Input	Result
01	Accept
001	Accept
101	Accept
111	Accept
0000	Accept
0110	Accept
1100	Accept
0101	Accept
110111	Accept
0	Reject
1	Reject
00	Reject
10	Reject
11	Reject
000	Reject
110	Reject
011	Reject
0010	Reject

Pruebas 154

155. *

Estado	0	1
$\rightarrow A$	B	C
B	D	E
C	K	G
D	H	E
E	F	D
$*F$	J	G
G	E	G
H	B	K
I	C	A
J	D	G
K	F	D
L	C	J

Razonamiento: Partiendo desde el estado inicial A, los caminos se ramifican hacia B, C, D, E, F, G, H, J y K. Sin embargo, los estados I y L jamás son alcanzados desde ninguna ruta válida. Por lo tanto, I y L son estados inaccesibles y deben eliminarse.

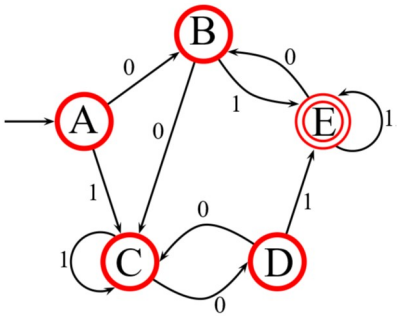


Autómata 155 (Simplificado)

Input	Result
010	Accept
100	Accept
0010	Accept
1100	Accept
0100010	Accept
010100	Accept
10110	Accept
11100	Accept
000010	Accept
0	Reject
1	Reject
00	Reject
11	Reject
01	Reject
10	Reject
000	Reject
0100	Reject
0101	Reject
	Reject

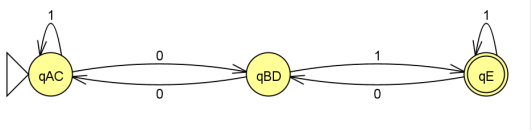
Pruebas 155

156. *



Simplifique el autómata encontrando una partición y/o eliminando los estados inaccesibles.

Razonamiento: Mediante el método de particiones, identificamos que los estados {A, C} se comportan igual y los estados {B, D} también. El autómata se minimiza a 3 estados combinados: qAC, qBD y qE.

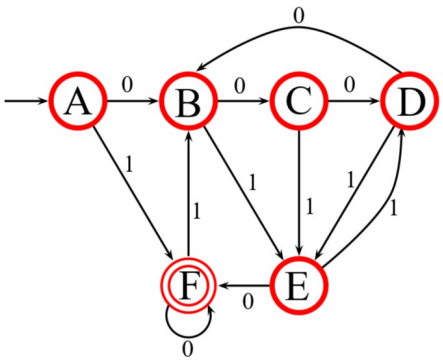


Autómata 156 (Minimizado)

Input	Result
01	Accept
101	Accept
011	Accept
0101	Accept
1101	Accept
0111	Accept
10101	Accept
0001	Accept
01011	Accept
0	Reject
1	Reject
00	Reject
10	Reject
010	Reject
110	Reject
100	Reject
000	Reject
0100	Reject

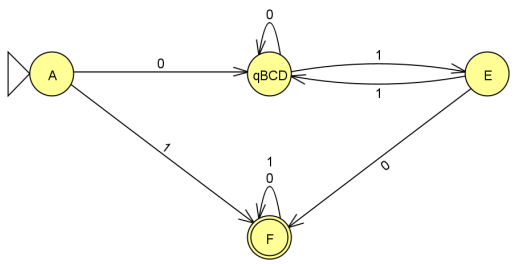
Pruebas 156

157. *



Simplifique el autómata encontrando una partición y/o eliminando los estados inaccesibles.

Razonamiento: Al aplicar particiones, los estados {B, C, D} resultan idénticos en comportamiento (todos transicionan a E con 1, y ciclan entre ellos con 0). El autómata se minimiza fusionándolos en un solo estado $qBCD$, conservando intactos a A, E y F.



Autómata 157 (Minimizado)

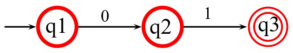
Input	Result
1	Accept
10	Accept
111	Accept
010	Accept
0010	Accept
01110	Accept
10101	Accept
001011	Accept
010000	Accept
0	Reject
00	Reject
01	Reject
001	Reject
011	Reject
000	Reject
0111	Reject
0011	Reject
000111	Reject

Pruebas 157

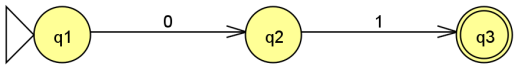
2. Autómatas Finitos No Deterministas

En los problemas 158-161, debe describir lo que hacen los autómatas finitos no deterministas definidos en cada tabla, donde el alfabeto es $\Sigma = \{0, 1\}$.

158. *



Razonamiento: El autómata acepta única y estrictamente la palabra "01".

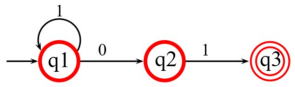


AFND 158

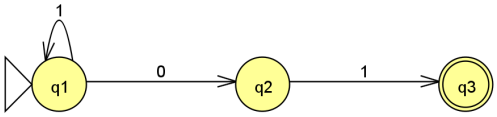
Input	Result
01	Accept
10	Reject
111	Reject
010	Reject
0010	Reject
01110	Reject
10101	Reject
001011	Reject
010000	Reject
0	Reject
00	Reject
00000	Reject
001	Reject
011	Reject
000	Reject
0111	Reject
0011	Reject
000111	Reject
	Reject

Pruebas 158

159. *



Razonamiento: El autómata acepta cadenas formadas por cualquier cantidad de unos (incluyendo cero unos) seguidos estrictamente por la subcadena "01".

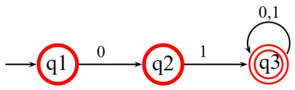


AFND 159

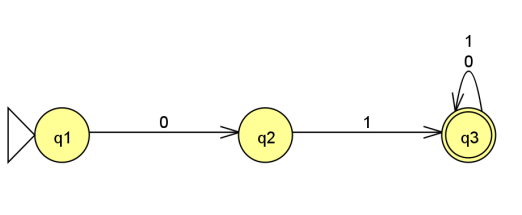
Input	Result
01	Accept
101	Accept
1101	Accept
11101	Accept
111101	Accept
1111101	Accept
11111101	Accept
111111101	Accept
1111111101	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
011	Reject
000	Reject
0111	Reject
0011	Reject
000111	Reject

Pruebas 159

160. *



Razonamiento: El autómata acepta cualquier cadena que comience con el prefijo "01"(y después puede seguir cualquier combinación de 0 y 1).

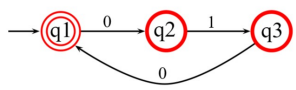


AFND 160

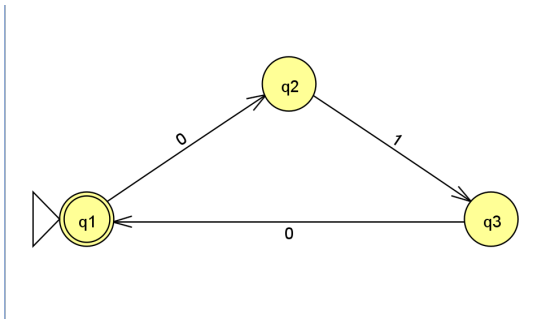
Input	Result
01	Accept
010	Accept
011	Accept
0100	Accept
0101	Accept
0110	Accept
0111	Accept
010101	Accept
011111	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
0011	Reject
000111	Reject

Pruebas 160

161. *



Razonamiento: El autómata acepta la cadena vacía (λ) y cualquier repetición exacta y consecutiva de la secuencia "010".



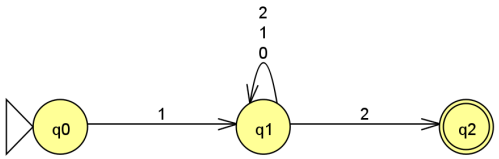
AFND 161

Input	Result
	Accept
010	Accept
010010	Accept
010010010	Accept
010010010010	Accept
010010010010010	Accept
010010010010010010	Accept
010010010010010010010	Accept
010010010010010010010010	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
0011	Reject
000111	Reject

Pruebas 161

162. * Encontrar un AFND que haga lo pedido en el problema 94 de la Tarea 2: Palabras de $\Sigma_1 = \{0, 1, 2\}$ que comienzan con 1 y terminan con 2.

Razonamiento: Se diseña un AFND con una transición inicial obligatoria con "1"hacia un estado con un lazo universal. El autómata .ªdivina.el final de la cadena mediante una transición con "2"hacia el estado de aceptación.



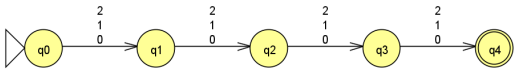
AFND 162

Input	Result
12	Accept
102	Accept
112	Accept
122	Accept
1002	Accept
1112	Accept
1212	Accept
1012	Accept
1222	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
0011	Reject
000111	Reject

Pruebas 162

163. * Encontrar un AFND que haga lo pedido en el problema 95 de la Tarea 2: Las palabras de Σ_1 que tienen longitud 4.

Razonamiento: Se construye una ruta lineal de cinco estados donde cada transición acepta cualquier símbolo del alfabeto. Al ser un AFND, no se requieren estados de trampa para longitudes mayores; la computación simplemente se detiene si no hay más transiciones.



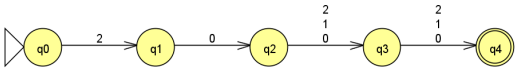
AFND 163

Input	Result
0000	Accept
1111	Accept
2222	Accept
0120	Accept
2101	Accept
1010	Accept
2020	Accept
1221	Accept
0012	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 163

164. * Encontrar un AFND que haga lo pedido en el problema 96 de la Tarea 2: Las palabras de Σ_1 con prefijo 20 y longitud 4.

Razonamiento: El autómata valida estrictamente el prefijo "2"seguido de "0.en los primeros dos pasos y posteriormente acepta dos caracteres cualesquiera para completar la longitud, omitiendo ramas de error.



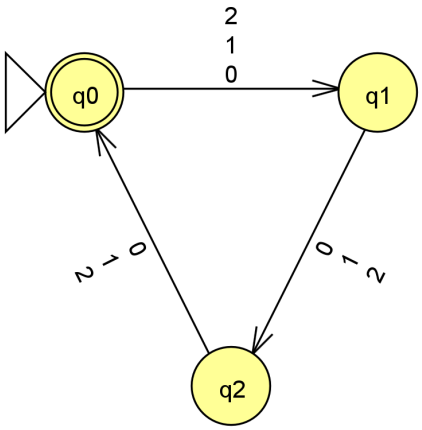
AFND 164

Input	Result
2000	Accept
2001	Accept
2002	Accept
2010	Accept
2011	Accept
2012	Accept
2020	Accept
2021	Accept
2022	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 164

165. * Encontrar un AFND que haga lo pedido en el problema 97 de la Tarea 2: Las palabras de Σ_1 cuya longitud es un múltiplo de 3.

Razonamiento: Se utiliza un ciclo de tres estados donde el estado inicial es de aceptación. Cada transición consume cualquier símbolo de Σ_1, regresando al estado inicial cada vez que la longitud es múltiplo de tres.



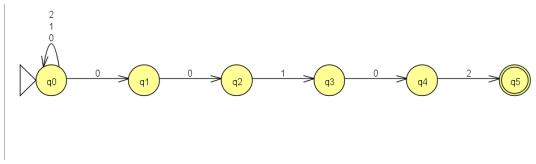
AFND 165

Input	Result
	Accept
000	Accept
111	Accept
222	Accept
012	Accept
000000	Accept
120120	Accept
222222	Accept
111111111	Accept
0	Reject
00	Reject
00000	Reject
001	Accept
000000	Accept
000	Accept
0000000000	Reject
00112220	Reject
000111	Accept

Pruebas 165

166. * Encontrar un AFND que haga lo pedido en el problema 98 de la Tarea 2: Las palabras de Σ_1 que terminan con la cadena 00102.

Razonamiento: Se coloca un lazo en el estado inicial con todo el alfabeto. De forma no determinista, el autómata inicia la secuencia "00102.^{en} el momento en que se presentan los últimos cinco símbolos de la cadena.



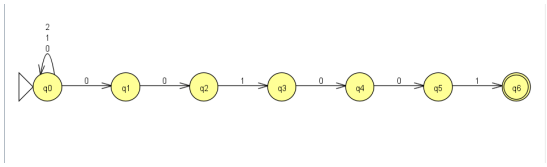
AFND 166

Input	Result
00102	Accept
100102	Accept
200102	Accept
000102	Accept
00100102	Accept
1200102	Accept
0010200102	Accept
22200102	Accept
10100102	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 166

167. * Encontrar un AFND que haga lo pedido en el problema 99 de la Tarea 2: Las palabras de Σ_1 que terminan con la cadena 001001.

Razonamiento: Similar al anterior, el autómata utiliza el no determinismo para saltar del lazo inicial hacia la rama que verifica estrictamente el sufijo "001001".



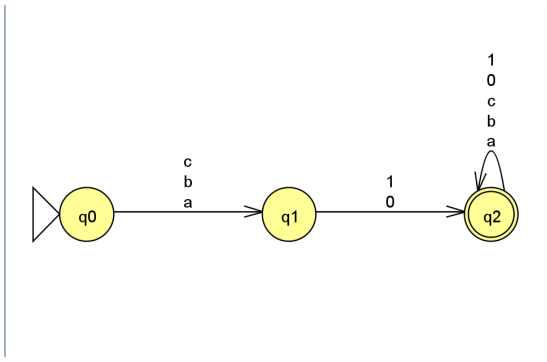
AFND 167

Input	Result
001001	Accept
1001001	Accept
2001001	Accept
0001001	Accept
001001001	Accept
12001001	Accept
01001001	Accept
222001001	Accept
0010001001	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 167

168. * Encontrar un AFND que haga lo pedido en el problema 100 de la Tarea 2: Las palabras de Σ_2 con letra en primer símbolo y dígito en el segundo.

Razonamiento: Se definen transiciones desde el estado inicial con las letras $\{a, b, c\}$ y desde el siguiente con dígitos $\{0, 1\}$. El tercer estado es de aceptación y permite cualquier combinación posterior del alfabeto Σ_2.



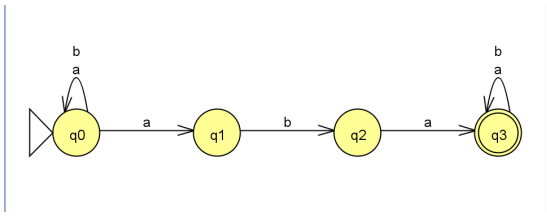
AFND 168

Input	Result
a0	Accept
b1	Accept
c0	Accept
a1abc	Accept
b0000	Accept
c1111	Accept
a0a	Accept
b1b	Accept
c0c0c0	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 168

169. * Encontrar un AFND que haga lo pedido en el problema 101 de la Tarea 2: Las palabras de Σ_3 que contienen la cadena aba.

Razonamiento: El AFND "busca"la subcadena mediante un lazo universal inicial, una ruta obligatoria .ª-b-az un lazo universal final tras la aceptación, simplificando los retrocesos del AFD.



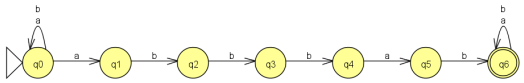
AFND 169

Input	Result
aba	Accept
aaba	Accept
baba	Accept
abaa	Accept
abab	Accept
ababa	Accept
bbaba	Accept
ababb	Accept
babab	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 169

170. * Encontrar un AFND que haga lo pedido en el problema 102 de la Tarea 2: Las palabras de Σ_3 que contienen la cadena abbbab.

Razonamiento: Se implementa una estructura de búsqueda de subcadena no determinista con la secuencia .ª-b-b-b-a-b.ᵉⁿtre dos estados con lazos para $\{a, b\}$.



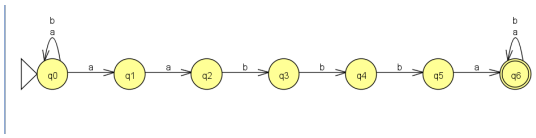
AFND 170

Input	Result
abbbab	Accept
aabbbab	Accept
babbbab	Accept
abbbaba	Accept
abbbabb	Accept
ababbbab	Accept
abbbabab	Accept
bbabbbab	Accept
aaabbbab	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 170

171. * Encontrar un AFND que haga lo pedido en el problema 103 de la Tarea 2: Las palabras de Σ_3 que contienen la cadena aabbba.

Razonamiento: Utilizando la misma lógica de subcadenas, el autómata acepta si en cualquier punto de la ejecución se completa la secuencia .ª-a-b-b-b-a".



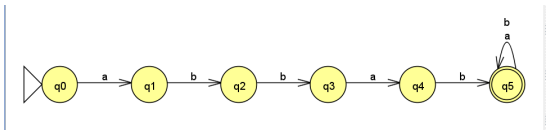
AFND 171

Input	Result
aabbba	Accept
aaabbba	Accept
baabbba	Accept
aabbbaa	Accept
aabbbab	Accept
ababaabbba	Accept
bbaabbba	Accept
aabbbaab	Accept
aabbbaaabbba	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 171

172. * Encontrar un AFND que haga lo pedido en el problema 104 de la Tarea 2: Las palabras de Σ_3 que inician con la cadena abbab.

Razonamiento: Se traza el camino del prefijo .ª-b-b-a-b"desde el inicio. Si se completa, se accede a un estado final con lazo universal; de lo contrario, la rama muere.



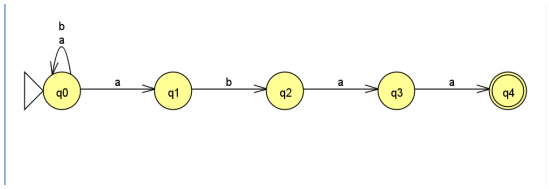
AFND 172

Input	Result
abbab	Accept
abbaba	Accept
abbabb	Accept
abbabaa	Accept
abbabab	Accept
abbabba	Accept
abbabbb	Accept
abbabaaa	Accept
abbabaabb	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 172

173. * Encontrar un AFND que haga lo pedido en el problema 105 de la Tarea 2: Las palabras de Σ_3 que terminan con la cadena abaa.

Razonamiento: El autómata utiliza un lazo inicial para consumir símbolos hasta que no determinísticamente decide iniciar la verificación del sufijo .ª-b-a-a".



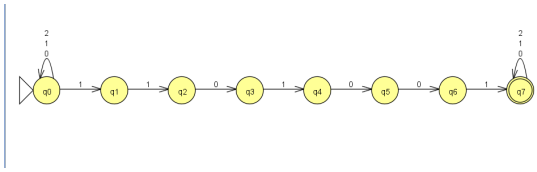
AFND 173

Input	Result
abaa	Accept
aabaa	Accept
babaa	Accept
ababaa	Accept
abaabaa	Accept
bbabaa	Accept
baabaa	Accept
aabaabaa	Accept
aaaaabaa	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 173

174. * Encontrar un AFND que haga lo pedido en el problema 106 de la Tarea 2: Las palabras de Σ_1 que incluyen la subcadena 1101001.

Razonamiento: Búsqueda de subcadena en Σ_1 mediante una ruta directa entre estados con lazos universales, eliminando la necesidad de estados de error.



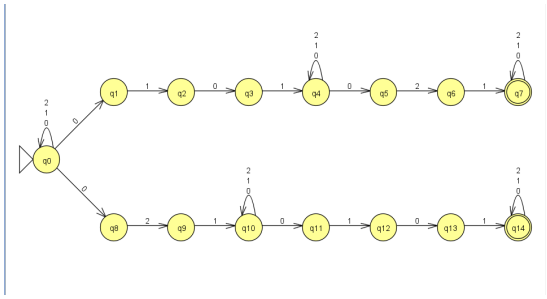
AFND 174

Input	Result
1101001	Accept
01101001	Accept
21101001	Accept
11010010	Accept
11010011	Accept
11010012	Accept
11101001	Accept
1101101001	Accept
20110100120	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 174

175. * Encontrar un AFND que haga lo pedido en el problema 107 de la Tarea 2: Palabras con subcadenas 0101 y 021.

Razonamiento: Se crean dos ramas principales desde el estado inicial: una busca el patrón A seguido del B, y la otra el patrón B seguido del A. Esto simplifica el producto de estados del AFD original.



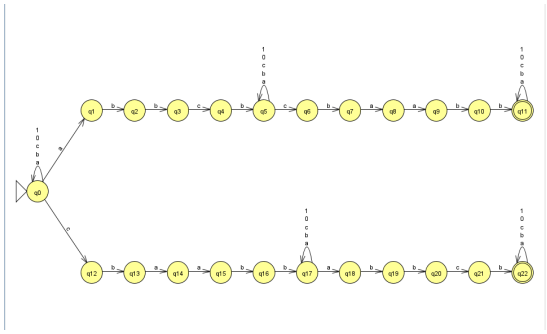
AFND 175

Input	Result
0101021	Accept
0210101	Accept
201012021	Accept
010122021	Accept
010101021	Accept
0210210101	Accept
110101220211	Accept
0212220101	Accept
01010210	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 175

176. * Encontrar un AFND que haga lo pedido en el problema 108 de la Tarea 2: Subcadenas abbcb y cbaabb sin traslape.

Razonamiento: Similar al anterior, pero en Σ_2. Las dos rutas posibles (A-B y B-A) aseguran que ambos patrones se encuentren de forma secuencial e independiente.



AFND 176

Input	Result
abbcbcbaabb	Accept
cbaabbabbcb	Accept
0abbbcb1cbaabb	Reject
abbbcbaaabbcbaabb	Reject
cbaabbcbaabbabbcb	Accept
01abbbcb10cbaabb	Reject
aabbbcbcbaabb	Reject
abbbcbcbaabb01	Reject
cbaabb01abbcb	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas 176

AFND: Ejercicios 177 al 188

Resumen del Razonamiento: En este bloque se aplican técnicas de bifurcación no determinista para contar ocurrencias de subcadenas (110-113), verificar estructuras de fase como $a^n b^m$ (114-118) y validar restricciones de patrones (119-120). La principal ventaja es la eliminación de estados de error y la simplificación de rutas concurrentes.

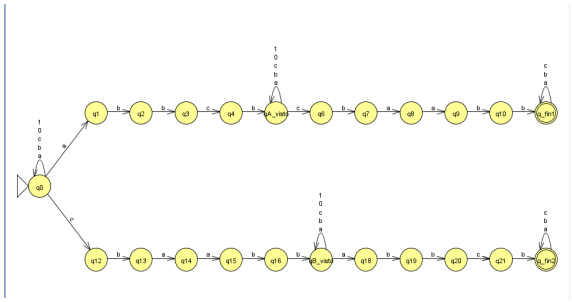


Diagrama e177

Input	Result
abbbcbaabb	Reject
cbaabbbcb	Accept
abbbcb01cbaabb	Reject
cbaabb01abbcb	Accept
00abbbcbaabb11	Reject
cbaabbcbaabbabbcb	Accept
abbbcbaaabbcbaabb	Reject
aaabbbcbcbaabbaa	Reject
01cbaabb10abbbcbc	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p177

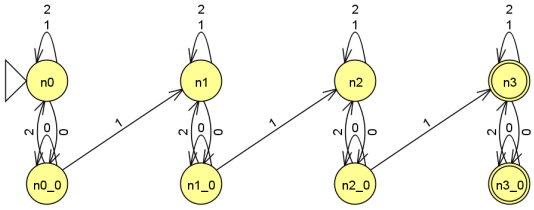


Diagrama e178

Input	Result
010101	Accept
2010101	Accept
01201201	Accept
01010122	Accept
001001001	Accept
220122012201	Accept
0111011101	Accept
0101001	Accept
1010101	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p178

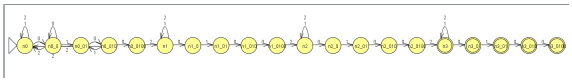


Diagrama e179

Input	Result
010020100201002	Accept
2010020100201002	Accept
0100222010021101002	Accept
0100201002010020	Accept
11010020100201002	Accept
01002010022201002	Accept
0100200100201002	Reject
01002101002201002	Accept
2220100201002010022 22	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p179

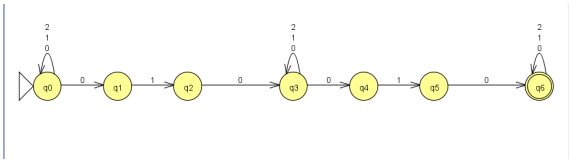


Diagrama e180

Input	Result
010010	Accept
0102010	Accept
0101010	Accept
2201022010	Accept
00100010	Accept
010111010	Accept
101021010	Accept
010010010	Accept
201020102	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p180

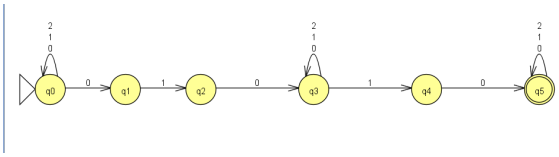


Diagrama e181

Input	Result
01010	Accept
010010	Accept
0101010	Accept
20101022	Accept
0102010	Accept
001010	Accept
110101011	Accept
010101010	Accept
2201010	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p181

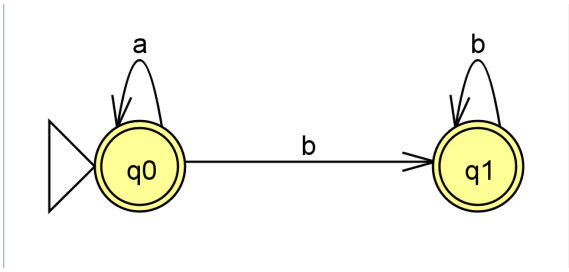


Diagrama e182

Input	Result
	Accept
a	Accept
b	Accept
aa	Accept
ab	Accept
bb	Accept
aaa	Accept
aabbb	Accept
aaaaabbbbb	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p182

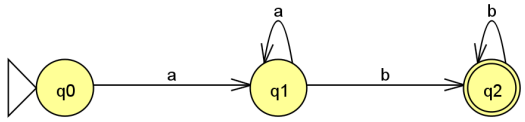


Diagrama e183

Input	Result
ab	Accept
aab	Accept
abb	Accept
aabb	Accept
aaabb	Accept
aabbb	Accept
aaaaab	Accept
abbbbb	Accept
aaaaabbbbb	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p183

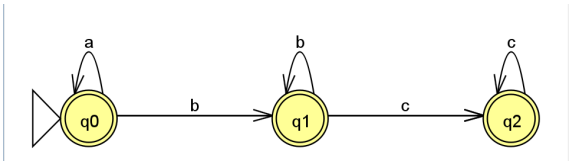


Diagrama e184

Input	Result
a	Accept
b	Accept
c	Accept
ab	Accept
ac	Accept
bc	Accept
abc	Accept
aaabbbccc	Accept
	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p184

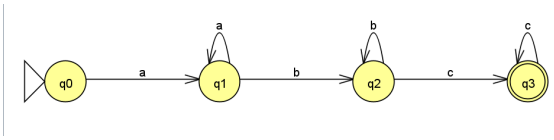


Diagrama e185

Input	Result
abc	Accept
aabc	Accept
abbc	Accept
abcc	Accept
aabbcc	Accept
aaabbbc	Accept
abccccc	Accept
aaaaabc	Accept
aaaaabbbbbccccc	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p185

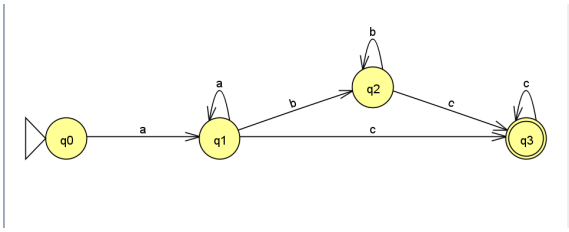


Diagrama e186

Input	Result
ac	Accept
abc	Accept
aac	Accept
acc	Accept
aabc	Accept
abcc	Accept
aabbcc	Accept
aaaccc	Accept
aaaaabbbbccccc	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p186

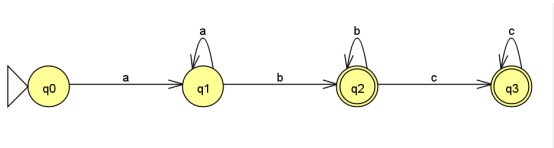


Diagrama e187

Input	Result
ab	Accept
abc	Accept
aab	Accept
abb	Accept
aaaaabbbbbccccc	Accept
aabb	Accept
abcc	Accept
aabbc	Accept
aaabbb	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p187

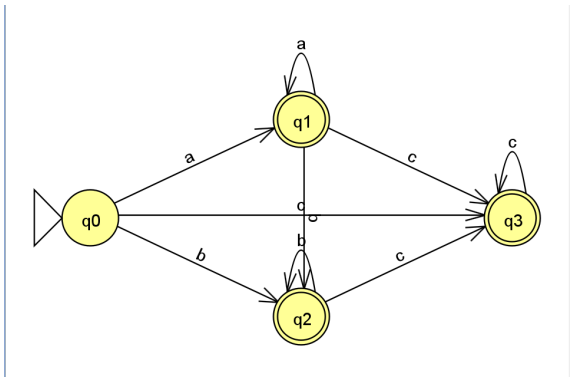


Diagrama e188

Input	Result
a	Accept
b	Accept
c	Accept
ab	Accept
ac	Accept
bc	Accept
abc	Accept
aaaaa	Accept
aaabbbccc	Accept
0	Reject
00	Reject
00000	Reject
001	Reject
000000	Reject
000	Reject
0000000000	Reject
00112220	Reject
000111	Reject

Pruebas p188

189. * Encuentre un autómata finito no determinista con lenguaje $\Sigma = \{0, 1\}$ que identifique las palabras que contienen la cadena 00101 o la cadena 00110. Realice el diagrama de transiciones de la palabra 1001011.

190. * Encuentre un autómata finito no determinista con lenguaje $\Sigma = \{0, 1\}$ que identifique las palabras que contienen la cadena 11010 y la cadena 10100. Realice el diagrama de transiciones de la palabra 01101010100.

191. * En los problemas 191-194, considere el autómata finito no determinista $A = (Q, \Sigma, \delta, A_0, F)$ definido en cada tabla. Encuentre un autómata finito determinista equivalente y realice las siguientes operaciones:

   a) Calcule $\delta^*(Z, \omega)$, donde δ es la función de transiciones del autómata.

   b) Si δ_D es la función de transición del autómata finito determinista asociado a A, calcule $\delta^*_D(Z, \omega)$.

192. * Considere el siguiente autómata no determinista:

Estado	0	1
$\rightarrow A$	B	B,C
B	A,B	$\emptyset$
$*C$	A,C	$\emptyset$

Realice las operaciones solicitadas:

a) $Z = \{A\}, \omega = 0111$

b) $Z = \{B, C\}, \omega = 010111$

193. * Considere el siguiente autómata no determinista:

Estado	0	1
$\rightarrow A$	$\emptyset$	B,C
B	A,D	B,C,D
C	C	A,C
$*D$	C	D

Realice las operaciones solicitadas:

a) $Z = \{A\}, \omega = 01101$

b) $Z = \{A, B\}, \omega = 1101$

c) $Z = \{C\}, \omega = 1101$

194. *

Estado	0	1	2
$\rightarrow A$	A,B	A,C	A
B	$\emptyset$	$\emptyset$	$\emptyset$
$*C$	A,B	A,B,C,D	$\emptyset$
$*D$	B	A	$\emptyset$

a) $Z = \{A, C\}, \omega = 0121$

b) $Z = \{B\}, \omega = 20111$

c) $Z = \{C\}, \omega = 02112$

d) $Z = \{A, B\}, \omega = 0121$

195. *

Estado	0	1
$\rightarrow A$	A,B	A
B	A	C
$*C$	A	D
D	D	E
$*E$	E	C,D

a) $Z = \{A\}, \omega = 10010$

b) $Z = \{A, D\}, \omega = 0110$

c) $Z = \{B, E\}, \omega = 0100$

d) $Z = \{B, C, D\}, \omega = 10100$

3. Autómatas finitos no deterministas con transiciones λ

Dibuje el diagrama de transiciones de cada uno de los autómatas que se proporcionan en los reactivos 196-200. Encuentre un autómata finito determinista equivalente a cada uno de los que se presentan en los ejercicios, proporcionando su tabla de transiciones y su diagrama de transiciones. Adicionalmente, halle la información que se pide en cada inciso, tomando en cuenta que δ es la función de transiciones del autómata original.

196.

- a) $\delta^*(A, 110)$.
- b) $\mathrm{Cl}(\{A\})$.
- c) $\mathrm{Cl}(\{B\})$.
- d) $\mathrm{Cl}(\{A, B\})$.
- e) $\delta(A, 0)$.

197.

- a) $\mathrm{Cl}(\{A\})$.
- b) $\mathrm{Cl}(\{B, D\})$.
- c) $\mathrm{Cl}(\{A, B, C\})$.
- d) $\delta(A, 0)$.
- e) $\delta^*(A, 010)$.

198.

- a) $\mathrm{Cl}(\{A\})$.
- b) $\mathrm{Cl}(\{C, D\})$.
- c) $\mathrm{Cl}(\{A, C, D\})$.
- e) $\delta^*(B, 0111)$.

199.

- a) Cl($\{A\}$).
- b) Cl($\{C\}$).
- c) $\delta^*(A, 011)$.

200.

- a) Cl($\{A\}$).
- b) Cl($\{C, D\}$).
- c) $\delta^*(C, 1011)$.

4. Expresiones regulares

Indique si las afirmaciones 201-238 son verdaderas (V) o falsas (F).

201. (V) $a \in L(a+b)$
202. (F) $a \in L(ab)$
203. (V) $a \in L(a^* + b^*)$
204. (F) $a \in L(ab^*)$
205. (F) $a \in L(a^*b)$
206. (F) $ab \in L(a+b)$
207. (V) $ab \in L(ab)$
208. (V) $ab \in L(\lambda a^*b)$
209. (F) $ab \in L(\emptyset a^*b)$
210. (V) $ab \in L((a+b)^*)$
211. (V) $ab \in L(abc^*)$
212. (F) $ab \in L(ab^*c)$
213. (V) $ab \in L(ab^*c^*)$
214. (V) $ab \in L((a+b+c)^*)$
215. (V) $ab \in L((abc)^*)$
216. (F) $ab \in L((ab)^*c)$
217. (V) $ab \in L(a(bc)^*)$
218. (V) $ab \in L(a(b+c)^*)$
219. (F) $ab \in L((a+b)^*c)$
220. (V) $ab \in L((a+b)^*c^*)$
221. (F) $aa \in L(a+b)$
222. (V) $aa \in L(a^* + b^*)$
223. (V) $aa \in L(a^*b^*)$
224. (V) $aa \in L((a+b)^*)$
225. (V) $aa \in L(a(a+b)^*)$
226. (V) $\lambda \in L(a)$
227. (V) $\lambda \in L((a+b)^*)$
228. (V) $\lambda \in L((a+b)^* + a)$
229. (F) $\lambda \in L(\emptyset)$
230. (V) $\lambda \in L(\lambda)$
231. (V) $\lambda \in L(\lambda + a)$
232. (V) $\lambda \in L(\lambda a)$
233. (V) $\lambda \in L(ab^*)$
234. (V) $\lambda \in L((ab)^*)$
235. (F) $\emptyset \in L(\emptyset)$

Razonamiento:

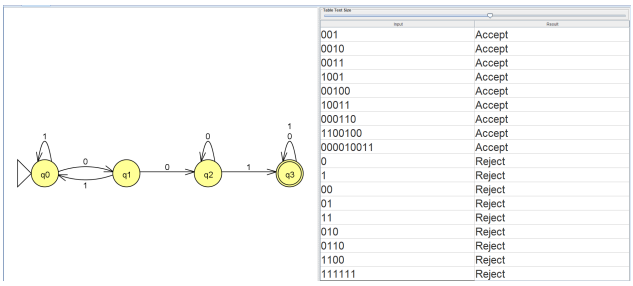
- 1. Unión L(a+b) = {a,b}. Solo acepta UNA opción. No mezcla ni concatena.
- 2. Concatenación L(ab) = {ab}. Debe aparecer exactamente en ese orden. No faltan ni sobran letras.
- 3. Cerradura de Kleene L(a^*) = {λ,a,aa,aaa,...}. SIEMPRE contiene λ (Puede repetir 0 o más veces).
- 4. Mezclas importantes $(a+b)^*$. a y b. Incluye λ.
- 5. Otros casos $\emptyset \in L(\emptyset)$. Falso. $\emptyset$ no es cadena, es un conjunto.

Para cada uno de los lenguajes L dados en 236-240, encuentre una expresión regular cuyo lenguaje sea L. En todos los casos, considere el alfabeto $\Sigma = \{0, 1\}$.

- 236. * $L = \{x001y : x, y \in \Sigma^*\}$.

  Razonamiento: Se busca cualquier cadena que contenga la subcadena `001`. Antes de `001` puede haber cualquier cadena sobre Σ (representada por $\Sigma^* = (0|1)^*$) y después también.

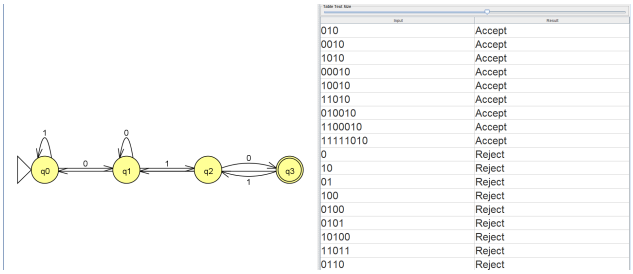
$$\boxed{(0|1)^*\,001\,(0|1)^*}$$



- 237. * $L = \{x010 : x \in \Sigma^*\}$.

  Razonamiento: Todas las cadenas que terminan con el sufijo `010`. Antes del sufijo puede ir cualquier cadena (incluyendo la vacía), por lo que la expresión regular es simplemente $(0|1)^*$ seguido del sufijo fijo `010`.

$$\boxed{(0|1)^*\,010}$$



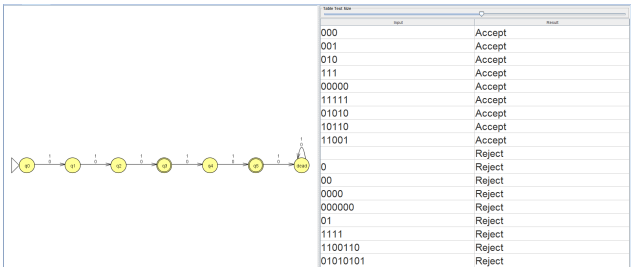
- 238. * $L = \Sigma^3 \cup \Sigma^5$.

  Razonamiento: El lenguaje acepta exactamente cadenas de **longitud 3** o **longitud 5**. Una cadena de longitud n sobre $\{0, 1\}$ se genera con $(0|1)^n$. La unión de ambos conjuntos:

$$\boxed{(0|1)^3 \mid (0|1)^5}$$

  lo que equivale a

$$(0|1)(0|1)(0|1) \mid (0|1)(0|1)(0|1)(0|1)(0|1)$$



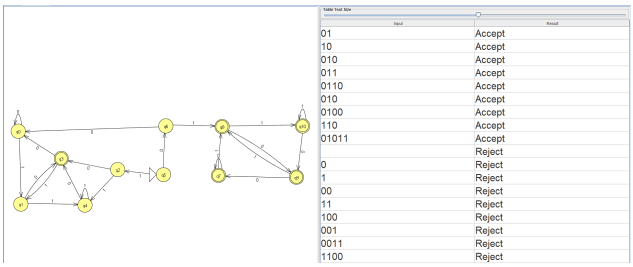
- 239. * L es el lenguaje de las cadenas que comienzan con 01 o que terminan con 10.

  Razonamiento: Se tienen dos condiciones que se unen con | (unión):

- Comienza con `01`: $01(0|1)^*$
- Termina con `10`: $(0|1)^*10$

$$\boxed{01(0|1)^* \;\Big|\; (0|1)^*10}$$

Nota: Las cadenas que cumplen ambas condiciones a la vez también pertenecen al lenguaje (p. ej., `0110` comienza con `01` y termina con `10`).



Input	Result
01	Accept
10	Accept
010	Accept
011	Accept
0110	Accept
010	Accept
0100	Accept
110	Accept
01011	Accept
	Reject
0	Reject
1	Reject
00	Reject
11	Reject
100	Reject
001	Reject
0011	Reject
1100	Reject

- 240. * L es el lenguaje de las cadenas que tienen longitud par y que contienen la subcadena 01.

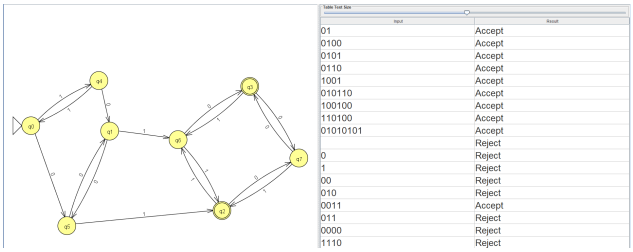
  Razonamiento: Combinamos dos condiciones:

  1. La cadena tiene longitud par: esto se modela con $((0|1)(0|1))^*$.
  2. La cadena contiene `01`: se necesita $(0|1)^*01(0|1)^*$.

  Una forma estándar es reconocer que se necesita un prefijo de longitud par + `01` + sufijo que complete la paridad:

$$\boxed{((0|1)(0|1))^*\,01\,((0|1)(0|1))^* \;\Big|\; (0|1)\,01\,(0|1)((0|1)(0|1))^*!}$$

  El primer término cubre los casos donde el prefijo hasta `01` tiene longitud par; el segundo donde el prefijo tiene longitud impar (un símbolo antes de `01` y un símbolo después, más pares adicionales).

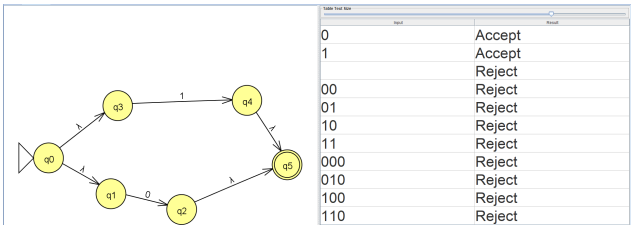


Input	Result
01	Accept
0100	Accept
0101	Accept
0110	Accept
1001	Accept
010110	Accept
100100	Accept
110100	Accept
01010101	Accept
	Reject
0	Reject
1	Reject
00	Reject
010	Reject
0011	Accept
011	Reject
0000	Reject
1110	Reject

En cada una de las expresiones regulares dadas en los problemas 241-253, encuentre un AFN-λ que sea equivalente a ésta.

- 241. $0 + 1$.

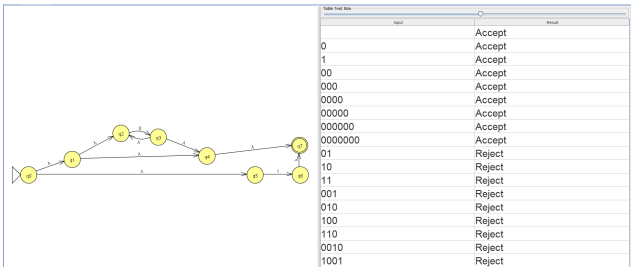
  Razonamiento: Unión de dos símbolos. Se crea un estado inicial q_0 y uno de aceptación q_5, con λ-transiciones hacia los subautómatas de `0` y `1`, y λ-transiciones de cada fin hacia q_5.



Input	Result
0	Accept
1	Accept
	Reject
00	Reject
01	Reject
10	Reject
11	Reject
000	Reject
010	Reject
100	Reject
110	Reject

- 242. 0* + 1.

  Razonamiento: Unión de 0^* y 1. Primero se construye 0^* con la regla de cerradura (acepta $\varepsilon, 0, 00, \ldots$), luego el símbolo 1, y finalmente se unen con la regla de unión.



Input	Result
	Accept
0	Accept
1	Accept
00	Accept
000	Accept
0000	Accept
00000	Accept
000000	Accept
0000000	Accept
01	Reject
10	Reject
11	Reject
001	Reject
010	Reject
100	Reject
110	Reject
0010	Reject
1001	Reject

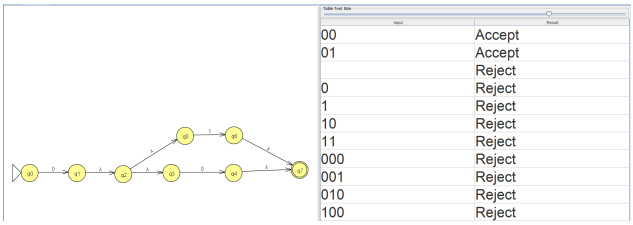
- 243. (0 + 1)*.

  Razonamiento: Cerradura de Kleene de la unión $0 + 1$. Primero se construye el AFN-λ para $0 + 1$ (ejercicio 241), y luego se aplica la cerradura de Kleene.

Input	Result
	Accept
0	Accept
1	Accept
00	Accept
01	Accept
10	Accept
11	Accept
000	Accept
101	Accept

- 244. 0(0 + 1).

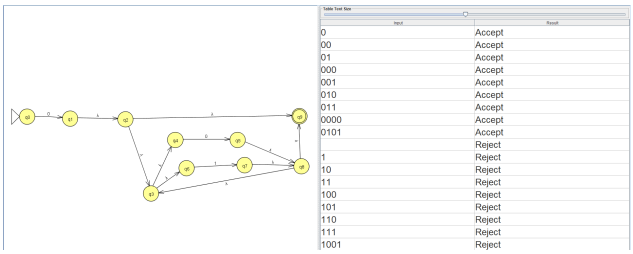
  Razonamiento: Concatenación del símbolo `0` con el AFN-λ de $0 + 1$. Se une el estado de aceptación del subautómata de `0` al estado inicial del subautómata de $0+1$ mediante una λ-transición. El lenguaje resultante es $\{00, 01\}$.



Input	Result
00	Accept
01	Accept
	Reject
0	Reject
1	Reject
10	Reject
11	Reject
000	Reject
001	Reject
010	Reject
100	Reject

- 245. 0(0 + 1)*.

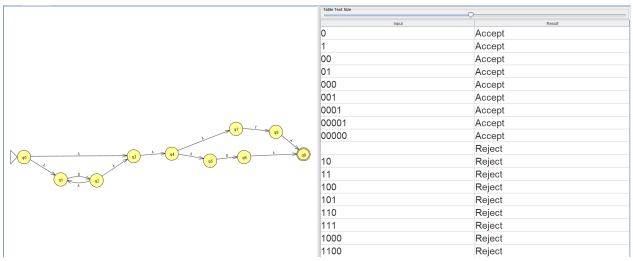
  Razonamiento: Concatenación de `0` con $(0+1)^*$. Todas las cadenas que comienzan con `0` seguidas de cualquier combinación de `0` y `1`.



Input	Result
0	Accept
00	Accept
01	Accept
000	Accept
001	Accept
010	Accept
011	Accept
0000	Accept
0101	Accept
	Reject
1	Reject
10	Reject
11	Reject
100	Reject
101	Reject
110	Reject
111	Reject
1001	Reject

- 246. 0*(0 + 1).

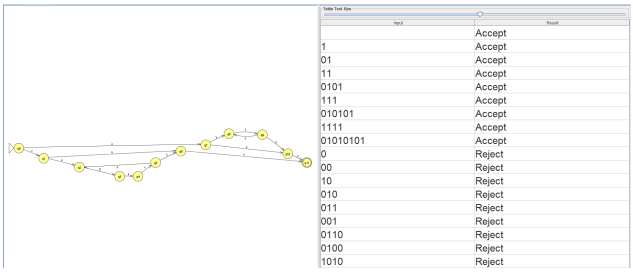
  Razonamiento: Concatenación de 0^* con $0+1$. Equivale a $0^*0+0^*1 = 0^+(0|1) \cup \{0, 1\}$, es decir, todas las cadenas no vacías sobre $\{0, 1\}$.



Input	Result
0	Accept
1	Accept
00	Accept
01	Accept
000	Accept
001	Accept
0001	Accept
00001	Accept
00000	Accept
	Reject
10	Reject
11	Reject
100	Reject
101	Reject
110	Reject
111	Reject
1000	Reject
1100	Reject

- 247. (01)* + 1*.

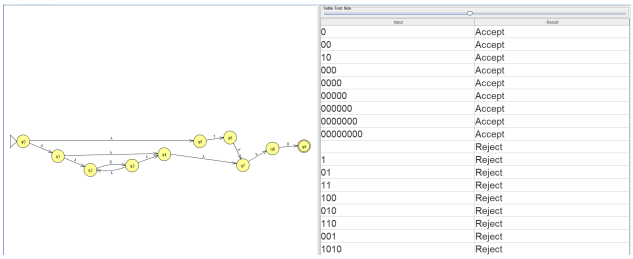
  Razonamiento: Unión de dos cerraduras. Se construye (01)*: cerradura de la concatenación de `0` y `1`; se construye 1*: cerradura de `1`; y se unen.



Input	Result
	Accept
1	Accept
01	Accept
11	Accept
0101	Accept
111	Accept
010101	Accept
1111	Accept
01010101	Accept
0	Reject
00	Reject
10	Reject
010	Reject
011	Reject
001	Reject
0110	Reject
0100	Reject
1010	Reject

- 248. (0* + 1)0.

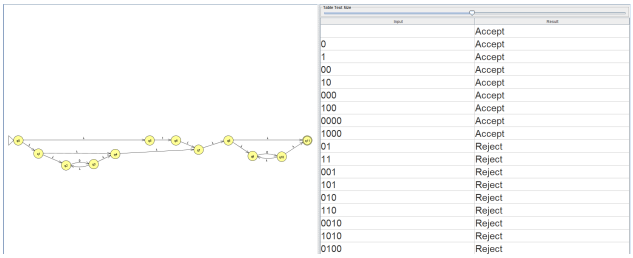
  Razonamiento: Concatenación de $(0^* + 1)$ con `0`. El lenguaje es el conjunto de cadenas que terminan en `0` y cuyo prefijo pertenece a $0^* + 1$, es decir: $0^*0 \cup \{10\}$.



Input	Result
0	Accept
00	Accept
10	Accept
000	Accept
0000	Accept
00000	Accept
000000	Accept
0000000	Accept
00000000	Accept
	Reject
1	Reject
01	Reject
11	Reject
100	Reject
010	Reject
110	Reject
001	Reject
1010	Reject

- 249. (0* + 1)0*.

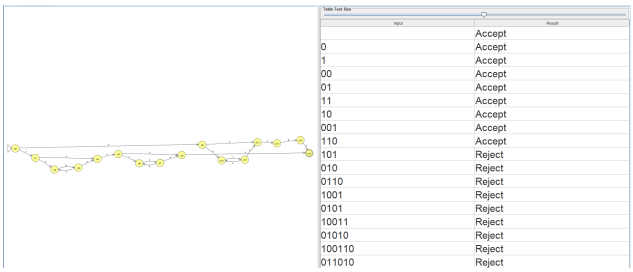
  Razonamiento: Concatenación de $(0^* + 1)$ con 0^*. El lenguaje es $0^* \cup 10^*$.



Input	Result
	Accept
0	Accept
1	Accept
00	Accept
10	Accept
000	Accept
100	Accept
0000	Accept
1000	Accept
01	Reject
11	Reject
001	Reject
101	Reject
010	Reject
110	Reject
0010	Reject
1010	Reject
0100	Reject

- 250. 0*1* + 1*0.

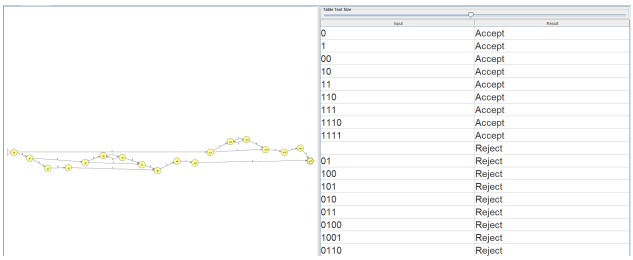
  Razonamiento: Unión de 0*1* y 1*0.



Input	Result
	Accept
0	Accept
1	Accept
00	Accept
01	Accept
11	Accept
10	Accept
001	Accept
110	Accept
101	Reject
010	Reject
0110	Reject
1001	Reject
0101	Reject
10011	Reject
01010	Reject
100110	Reject
011010	Reject

- 251. $(0 + 1^*)0 + 1^*1$.

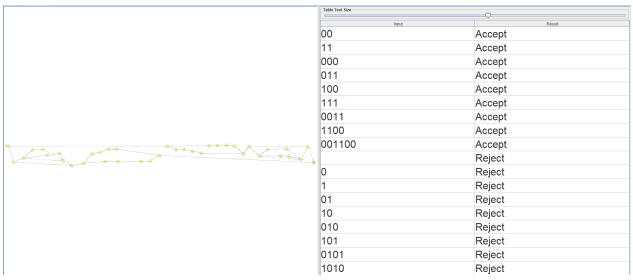
  Razonamiento: Unión de dos concatenaciones. Da: `00` y 1^*0 por un lado, y 1^+ por el otro.



Input	Result
0	Accept
1	Accept
00	Accept
10	Accept
11	Accept
110	Accept
111	Accept
1110	Accept
1111	Accept
	Reject
01	Reject
100	Reject
101	Reject
010	Reject
011	Reject
0100	Reject
1001	Reject
0110	Reject

- 252. $(0 + 1)^*(00 + 11) + (00 + 11)(0 + 1)^*$.

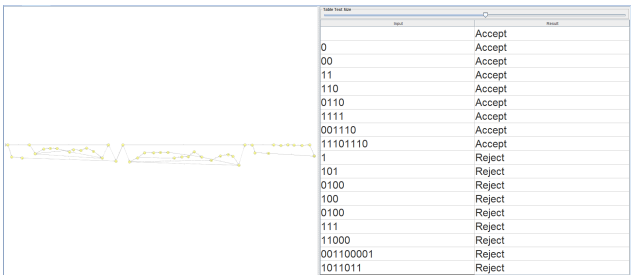
  Razonamiento: Unión de dos partes. Cadenas que terminan con `00` o `11`, o cadenas que comienzan con `00` o `11`. En la construcción de Thompson se generan ambas ramas y se unen con un nuevo estado inicial y uno final.



Input	Result
00	Accept
11	Accept
000	Accept
011	Accept
100	Accept
111	Accept
0011	Accept
1100	Accept
001100	Accept
	Reject
0	Reject
1	Reject
01	Reject
10	Reject
010	Reject
101	Reject
0101	Reject
1010	Reject

- 253. $(0 + (11 + 00)^*)(11 + 10 + 01)^*(\lambda + 110)$.

  Razonamiento: Concatenación de tres partes. La construcción de Thompson aplica concat(concat(parte1, parte2), parte3).



Input	Result
	Accept
0	Accept
00	Accept
11	Accept
110	Accept
0110	Accept
1111	Accept
001110	Accept
11101110	Accept
1	Reject
101	Reject
0100	Reject
100	Reject
0100	Reject
111	Reject
11000	Reject
001100001	Reject
1011011	Reject

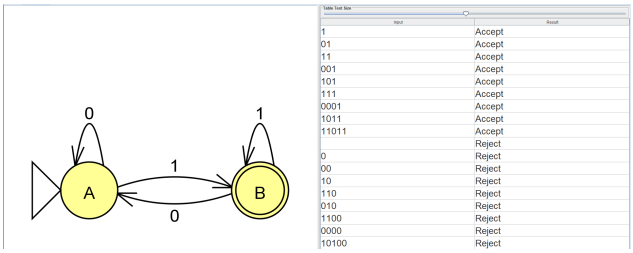
Para cada autómata que se presenta en los reactivos 254-258, encuentre una expresión regular que lo represente.

- 254. *

Estado	0	1
$\rightarrow A$	A	B
$*B$	A	B

Razonamiento: Sea $S = A + B$. Notamos que A y B tienen las mismas transiciones salvo el origen; en ambos estados con 0 volvemos a A y con 1 vamos a B. El autómata acepta todas las cadenas que terminan en 1.

$$\boxed{(0|1)^*\ 1}$$

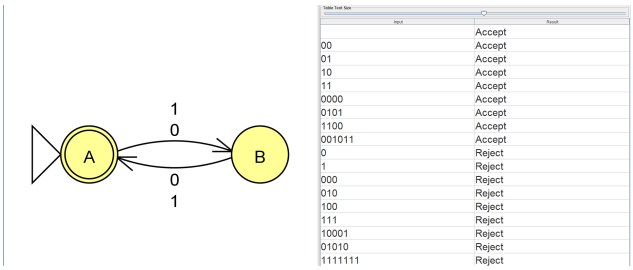


- 255. *

Estado	0	1
$\rightarrow *A$	B	B
$*B$	A	B

Razonamiento: El estado inicial A también es de aceptación. Al leer cualquier símbolo desde A pasamos a B. Desde B: $\texttt{0} \rightarrow A$, $\texttt{1} \rightarrow B$. La paridad de la longitud determina el estado. Longitud par $\Rightarrow$ estado $A \Rightarrow$ aceptado.

$$\boxed{((0|1)(0|1))^*}$$



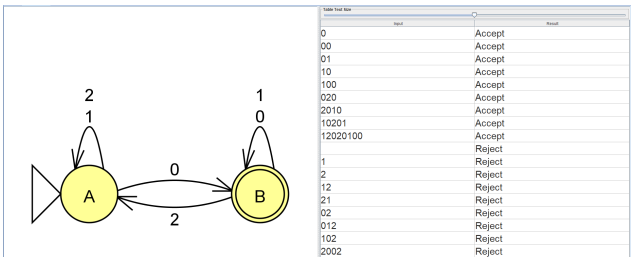
- 256. *

Estado	0	1	2
$\rightarrow A$	B	A	A
$*B$	B	B	A

Razonamiento: Usando ecuaciones de Arden:

$$A = \varepsilon + A \cdot 1 + A \cdot 2 + B \cdot 2$$
$$B = A \cdot 0 + B \cdot 0 + B \cdot 1$$

$$\boxed{(1|2|0(0|1)^*2)^*\ 0\ (0|1)^*}$$

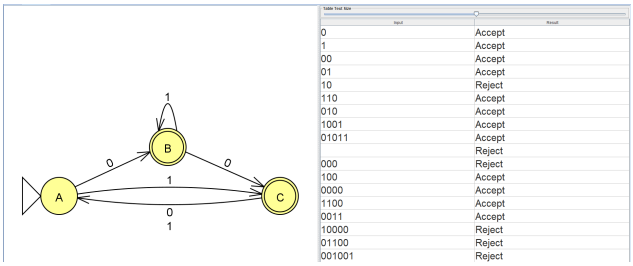


- 257. *

Estado	0	1
$\rightarrow A$	B	C
$*B$	C	B
$*C$	A	A

Razonamiento: Factorizando A:

$$\boxed{((1|01^*0)(0|1))^* \ (01^* \mid 1 \mid 01^*0)}$$

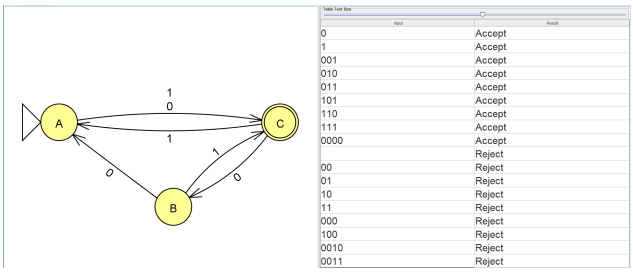


- 258. *

Estado	0	1
$\rightarrow A$	C	C
B	A	C
$*C$	B	A

Razonamiento: Por lema de Arden se obtiene que las cadenas aceptadas terminan en C:

$$\boxed{((0|1)(01)^*0)^* \ (0|1)(01)^*}$$

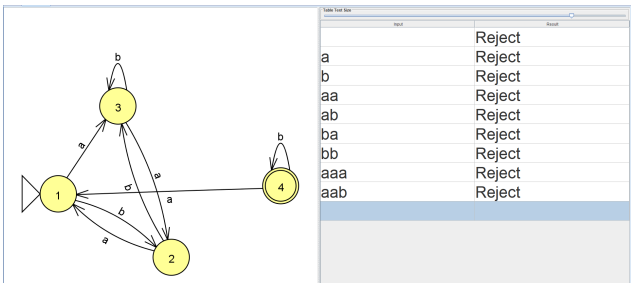


- 259. Considere el autómata de la siguiente tabla:

Estado	a	b
$\rightarrow 1$	3	2
2	1	3
3	2	3
$*4$	1	4

Razonamiento: Analizando el autómata: el único estado de aceptación es 4, y el estado 4 sólo es alcanzado desde sí mismo leyendo b. Por lo tanto el lenguaje aceptado es **vacío** ($L = \emptyset$), ya que no existe ninguna trayectoria del estado 1 al estado 4.

$$\boxed{L = \emptyset}$$



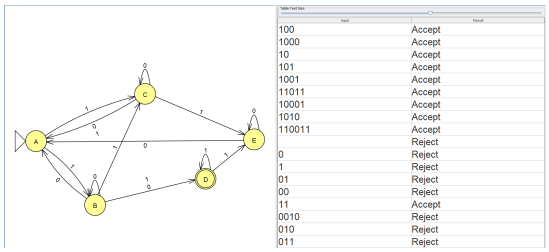
5. Equivalencia de autómatas finitos deterministas y no deterministas

260. * Encuentre un (AFD) que sea equivalente al siguiente autómata finito no determinista (AFN).

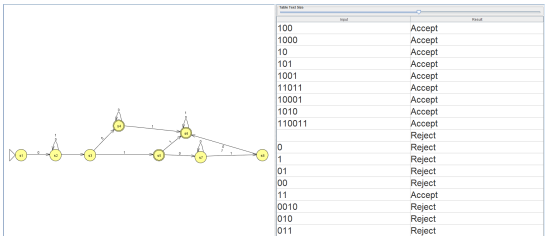
Estado	0	1
$\rightarrow A$	$\emptyset$	{B, C}
B	{A, B, D}	{C, D}
C	{A, C}	{A, E}
$*D$	$\emptyset$	{D, E}
E	{A, E}	$\emptyset$

Razonamiento: Partimos del estado inicial $\{A\}$ y calculamos las transiciones para cada subconjunto alcanzable. Los estados de aceptación del AFD son los subconjuntos que contienen a D. **Tabla del AFD resultante:**

	0	1
$\rightarrow s_1$	s_2	s_3
s_2	s_2	s_2
s_3	s_4	s_5
$*s_4$	s_4	s_6
$*s_5$	s_7	s_6
$*s_6$	s_6	s_6
s_7	s_7	s_8
s_8	s_6	s_6

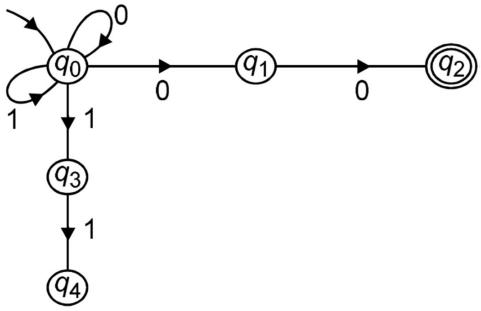


AFN 260



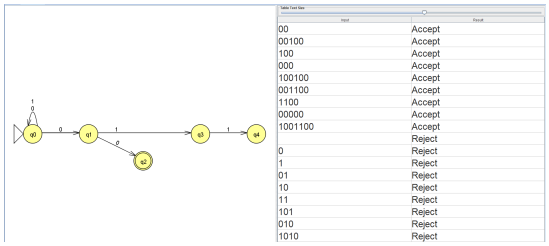
AFD 260

261. * Dado el autómata finito no determinista (AFN) mostrado en la siguiente figura, determine el autómata finito determinista (AFD) equivalente:

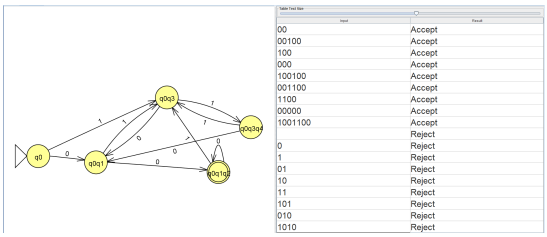


El AFN tiene estados q_0, q_1, q_2, q_3, q_4. Estados de aceptación: q_2. **Tabla del AFD:**

	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_3\}$
$\{q_0, q_3\}$	$\{q_0, q_1\}$	$\{q_0, q_3, q_4\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_3\}$
$\{q_0, q_3, q_4\}$	$\{q_0, q_1\}$	$\{q_0, q_3\}$



AFN 261

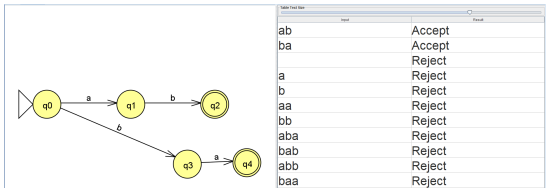


AFD 261

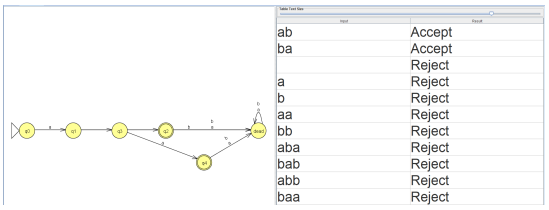
262. * Diseñe un (AFN) que acepte el conjunto {ab, ba} y utilícelo para encontrar un (AFD) que lo acepte.

Razonamiento: Se diseña un AFN con estados q_0 (inicial), q_1, q_2, q_3, q_4 (de aceptación). **Tabla del AFD:**

	a	b
$\rightarrow \{q_0\}$	$\{q_1\}$	$\{q_3\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_3\}$	$\{q_4\}$	$\emptyset$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$*\{q_4\}$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$

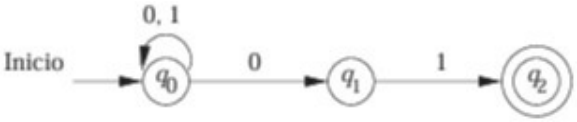


AFN 262



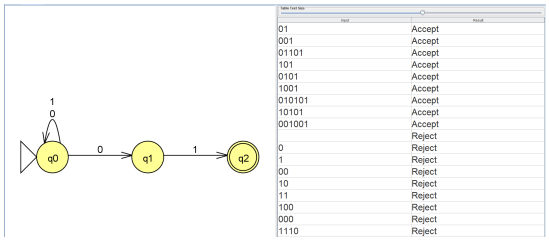
AFD 262

263. * Dado el autómata finito no determinista (AFN) mostrado en la siguiente figura, determine el autómata finito determinista (AFD) equivalente:

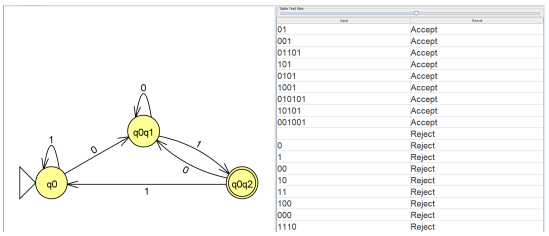


Del diagrama, el AFN tiene estados q_0, q_1, q_2. **Tabla del AFD:**

	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$



AFN 263



AFD 263